

DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	KUNKU ROHIT
ROLL NO	AV.SC.U4CSE25212
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	Dr Gayathri Parasa
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	5-9
2.	**WEEK 2:** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	10-12
3.	**WEEK-3:** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	13-15
4.	**WEEK-4:** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	16-19
5.	**WEEK-5:** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning	20-25

	3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10)Search data in SLL 11)Display or Traverse 12)Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10)Exit	26-31
7.	**WEEK-7:** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	32-37
8.	**WEEK-8:** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	38-47

9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	48-52
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	53-57
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	58-61
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	62-67

WEEK – 1

PROGRAM-1(A):

AIM: To write a recursive Python program to display a countdown from n to 1, followed by "Launch!".

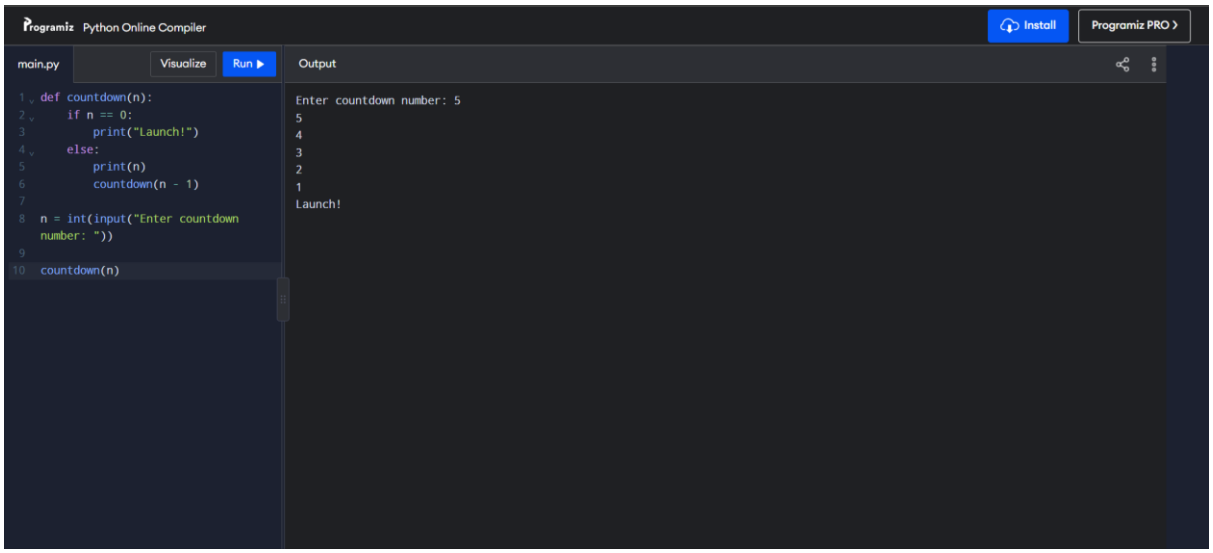
CODE:

```
def countdown(n):
    if n == 0:
        print("Launch!")
    else:
        print(n)
        countdown(n - 1)

n = int(input("Enter countdown number: "))

countdown(n)
```

OUTPUT:



PROGRAM-1(B):

AIM: To write a recursive Python program to calculate P^n (P = principal growth factor, n = number of years) for compound growth of an investment.

CODE:

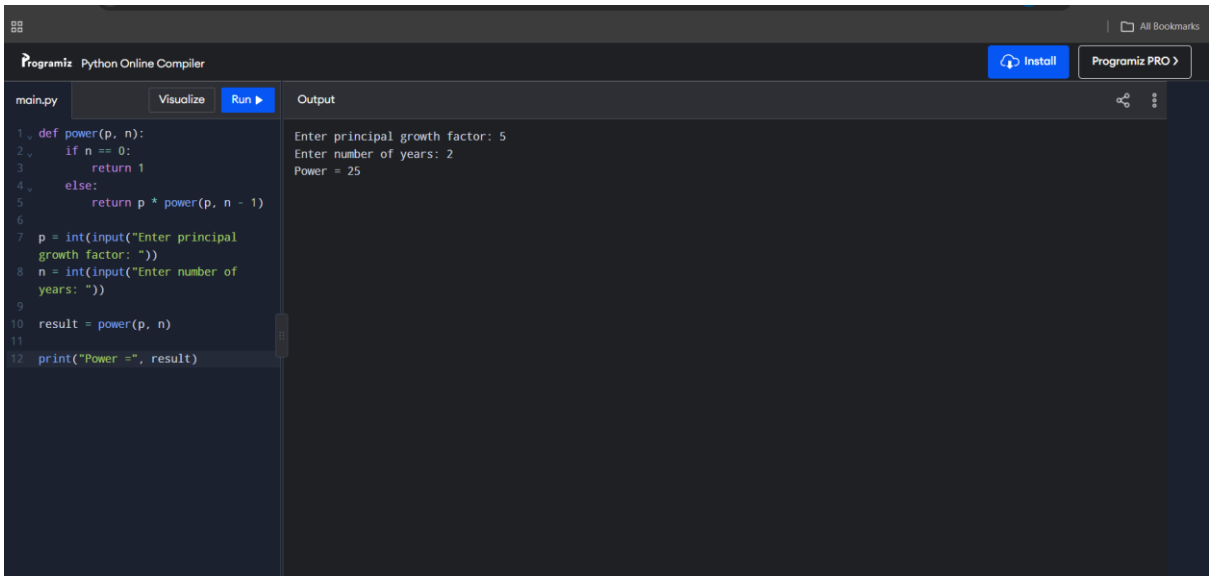
```
def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

p = int(input("Enter principal growth factor: "))
n = int(input("Enter number of years: "))

result = power(p, n)

print("Power =", result)
```

OUTPUT:



PROGRAM-1(C):

AIM: To write a recursive Python function to search for a given employee ID in a list.

CODE:

```
def search(arr, key, i):
    if i == len(arr):
        return -1

    if arr[i] == key:
        return i

    return search(arr, key, i + 1)

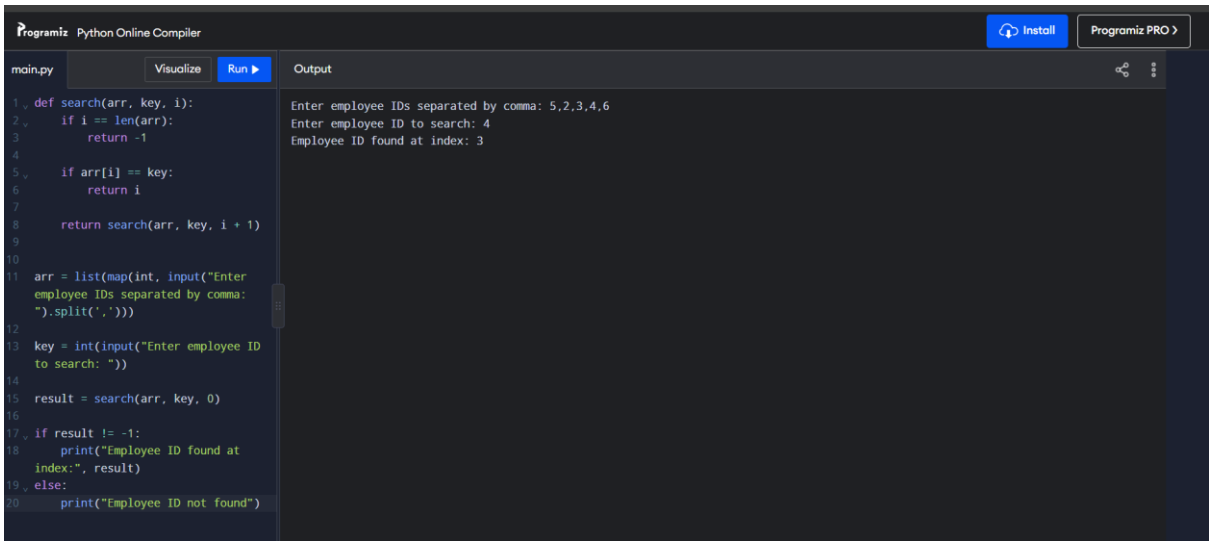

arr = list(map(int, input("Enter employee IDs separated by comma:
").split(',')))

key = int(input("Enter employee ID to search: "))

result = search(arr, key, 0)

if result != -1:
    print("Employee ID found at index:", result)
else:
    print("Employee ID not found")
```

OUTPUT:



PROGRAM-1(D):

AIM: To write a recursive Python program to calculate the factorial of a given number.

CODE:

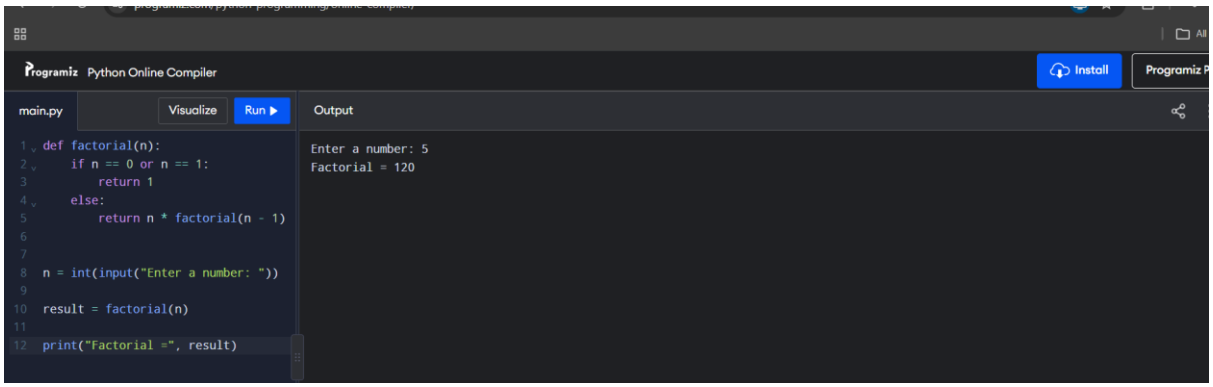
```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)


n = int(input("Enter a number: "))

result = factorial(n)

print("Factorial =", result)
```

OUTPUT:



PROGRAM-1(E):

AIM: To write a recursive Python program to print the first n terms of the Fibonacci series.

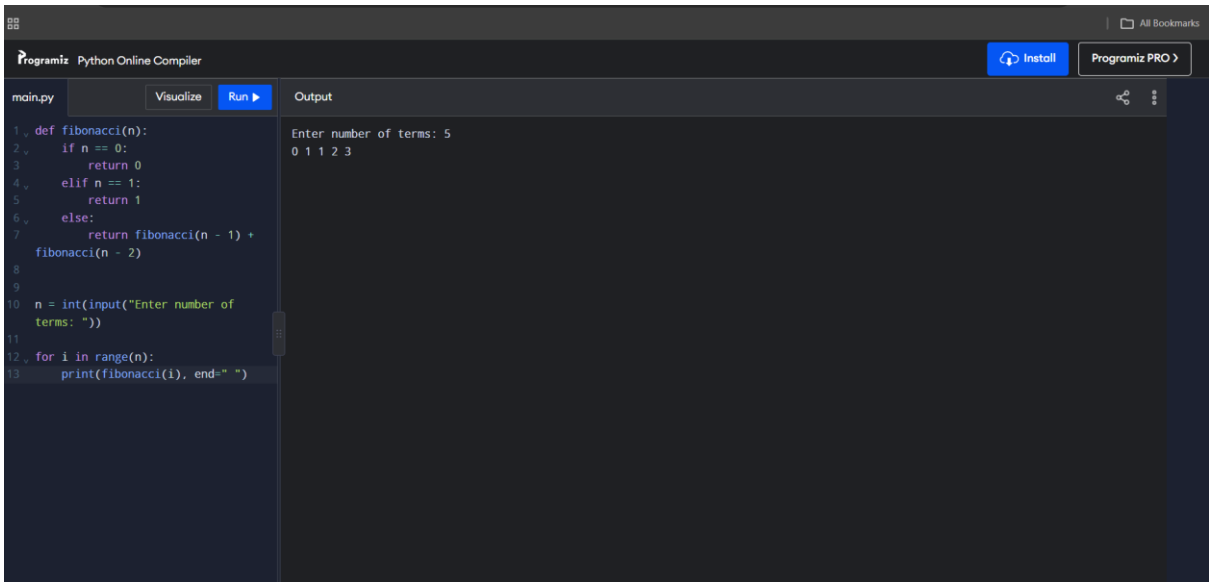
CODE:

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)


n = int(input("Enter number of terms: "))

for i in range(n):
    print(fibonacci(i), end=" ")
```

OUTPUT:



WEEK – 2

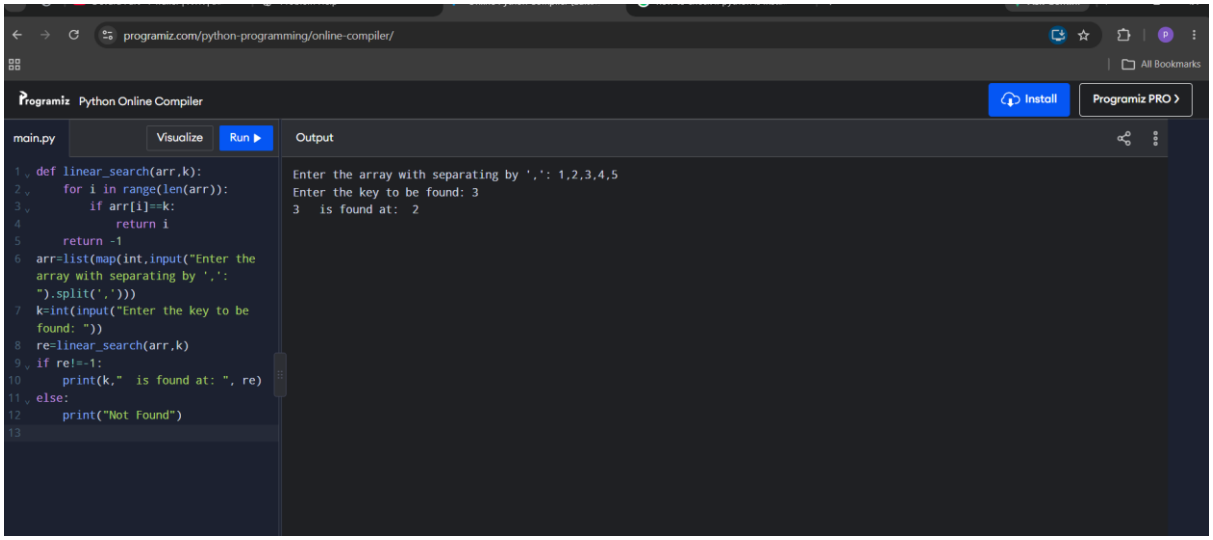
PROGRAM-2(A):

AIM: To write a Python program to implement linear search.

CODE:

```
def linear_search(arr,k):
    for i in range(len(arr)):
        if arr[i]==k:
            return i
    return -1
arr=list(map(int,input("Enter the array with separating by ',': ").split(',')))
k=int(input("Enter the key to be found: "))
re=linear_search(arr,k)
if re!=-1:
    print(k,"  is found at: ", re)
else:
    print("Not Found")
```

OUTPUT:



PROGRAM-2(B):

AIM: To write a Python program to implement binary search with and without a sorted array as an input.

CODE:

Binary Search (Iterative) – sorted array

```
def binary_search(arr,k):
    lo=0
    high=len(arr)-1
    while lo<=high:
        mid=(lo+high)//2
        if arr[mid]==k:
            return mid
        elif arr[mid]>k:
            high=mid-1
        else:
            lo=mid+1
    return -1
arr=list(map(int,input("Enter the array with separating by ',': ").split(',')))
k=int(input("Enter the key to be found: "))
re=binary_search(arr,k)
if re!=-1:
    print(k,"  is found at: ", re)
else:
    print("Not Found")
```

Binary Search (Recursive) – sorted array

```
def binary_search(lo,high,arr,k):
    if lo<=high:
        mid=(lo+high)//2
        if arr[mid]==k:
            return mid
        elif arr[mid]>k:
            return binary_search(lo,mid-1,arr,k)
        else:
            return binary_search(mid+1,high,arr,k)
    else:
        return -1
arr=list(map(int,input("Enter the array with separating by ',': ").split(',')))
k=int(input("Enter the key to be found: "))
re=binary_search(0,len(arr)-1,arr,k)
if re!=-1:
    print(k,"  is found at: ", re)
else:
    print("Not Found")
```

Binary Search with unsorted array as input

```
def binary_search(arr, k):
```

```
    arr.sort()

    lo = 0
    high = len(arr) - 1

    while lo <= high:

        mid = (lo + high) // 2

        if arr[mid] == k:
            return mid

        elif arr[mid] > k:
            high = mid - 1

        else:
            lo = mid + 1

    return -1


arr = list(map(int, input("Enter array separated by comma: ").split(',')))

k = int(input("Enter the key to be found: "))

re = binary_search(arr, k)

if re != -1:
    print(k, "is found at index:", re)
else:
    print("Not Found")
```

OUTPUT:



WEEK – 3

PROGRAM-3(A):

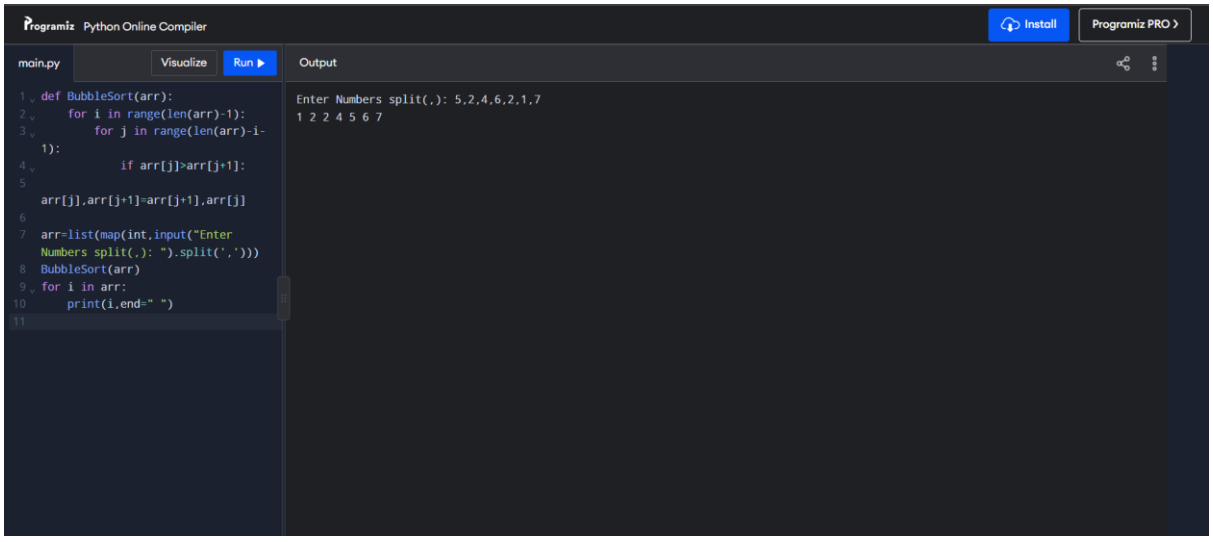
AIM: To write a Python program to implement bubble sort.

CODE:

```
def BubbleSort(arr):
    for i in range(len(arr)-1):
        for j in range(len(arr)-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]

arr=list(map(int,input("Enter Numbers split(,): ").split(',')))
BubbleSort(arr)
for i in arr:
    print(i,end=" ")
```

OUTPUT:



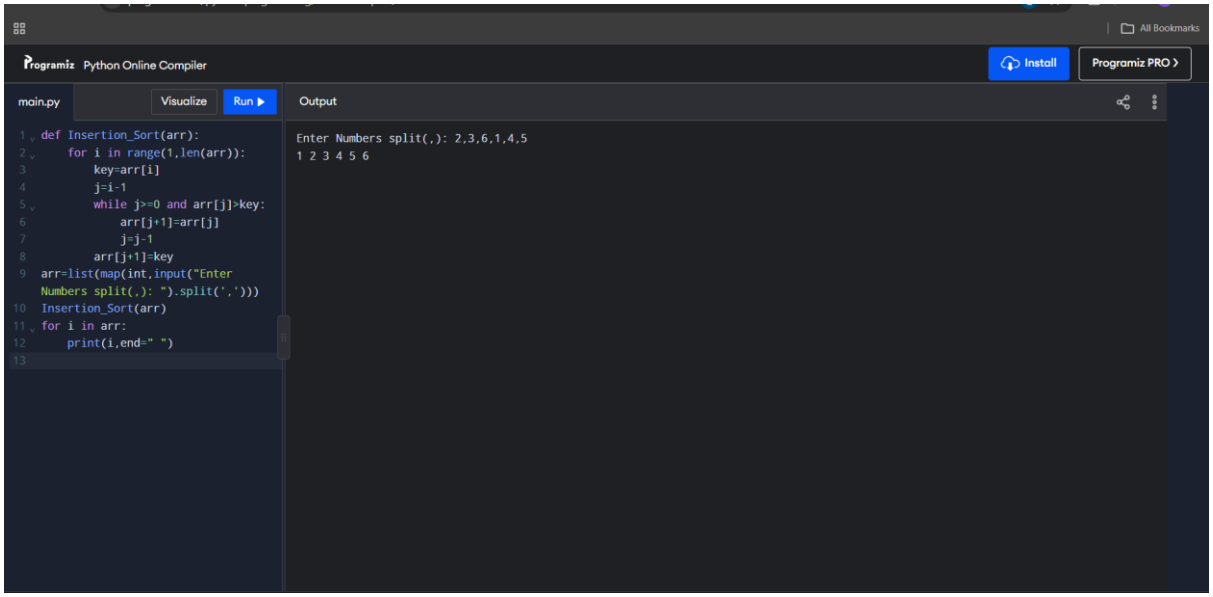
PROGRAM-3(B):

AIM: To write a Python program to implement insertion sort.

CODE:

```
def Insertion_Sort(arr):
    for i in range(1,len(arr)):
        key=arr[i]
        j=i-1
        while j>=0 and arr[j]>key:
            arr[j+1]=arr[j]
            j=j-1
        arr[j+1]=key
arr=list(map(int,input("Enter Numbers split(,): ").split(',')))
Insertion_Sort(arr)
for i in arr:
    print(i,end=" ")
```

OUTPUT:



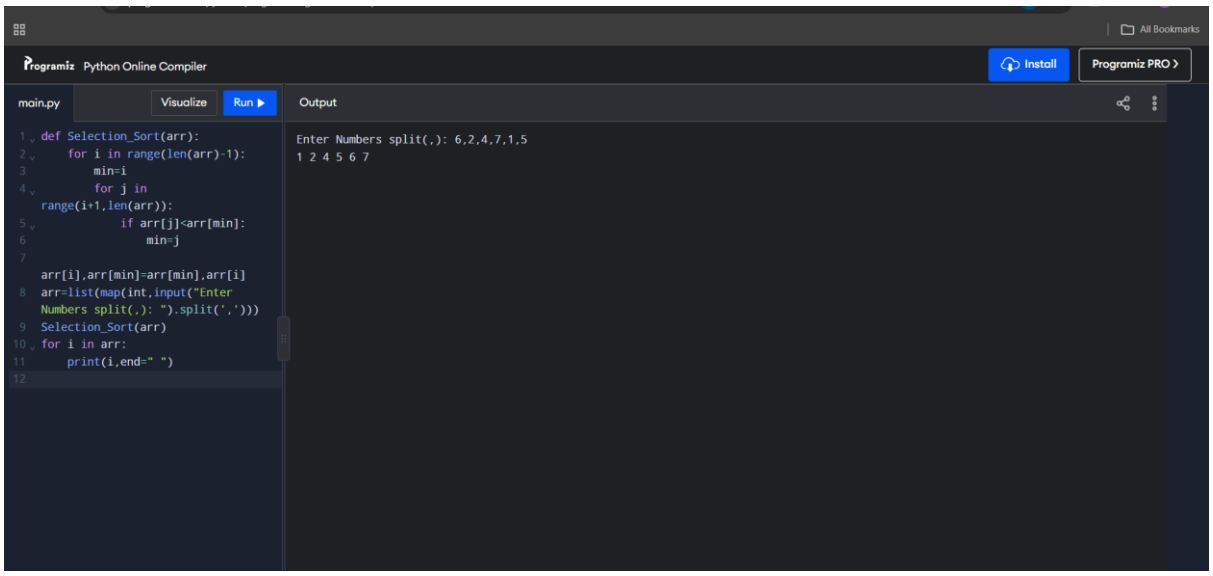
PROGRAM-3(C):

AIM: To write a Python program to implement selection sort.

CODE:

```
def Selection_Sort(arr):
    for i in range(len(arr)-1):
        min=i
        for j in range(i+1,len(arr)):
            if arr[j]<arr[min]:
                min=j
        arr[i],arr[min]=arr[min],arr[i]
arr=list(map(int,input("Enter Numbers split(,): ").split(',')))
Selection_Sort(arr)
for i in arr:
    print(i,end=" ")
```

OUTPUT:



WEEK – 4

PROGRAM-4(A):

AIM: To write a Python program to implement quick sort.

CODE:

```
def quicksort(a, low, high):

    if low < high:
        i = low
        j = high
        pivot = low

        while i < j:

            while i < len(a) and a[i] <= a[pivot]:
                i += 1

            while a[j] > a[pivot]:
                j -= 1

            if i < j:
                a[i], a[j] = a[j], a[i]

        a[j], a[pivot] = a[pivot], a[j]

        quicksort(a, low, j - 1)
        quicksort(a, j + 1, high)

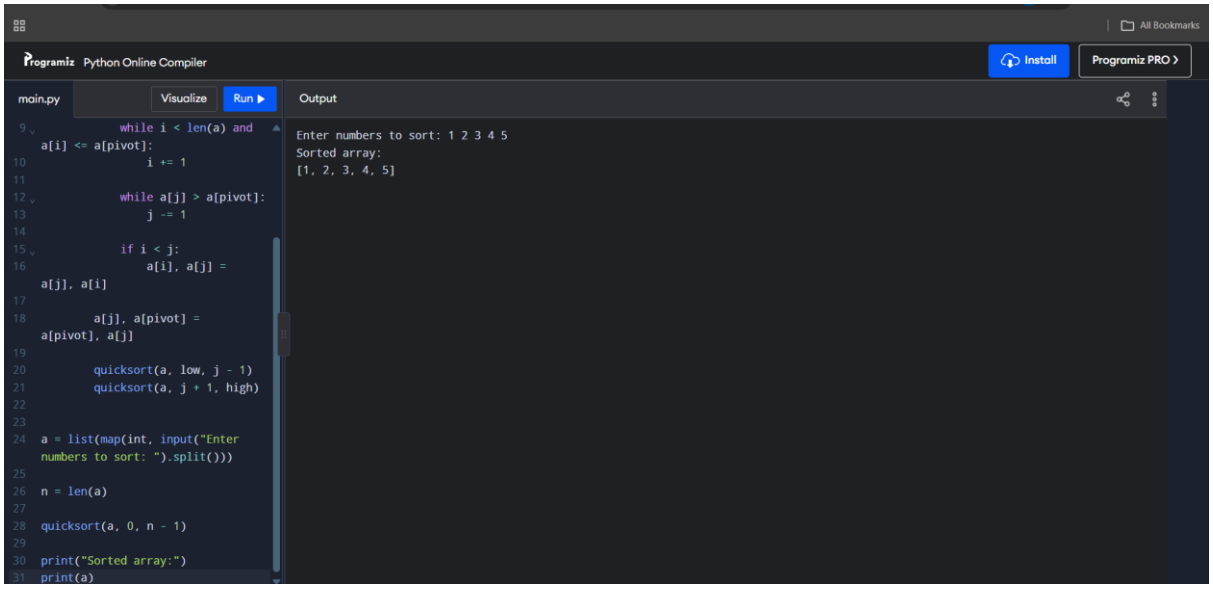

a = list(map(int, input("Enter numbers to sort: ").split()))

n = len(a)

quicksort(a, 0, n - 1)

print("Sorted array:")
print(a)
```

OUTPUT:


All Bookmarks
Programiz Python Online Compiler
Install
Programiz PRO
main.py
Visualize
Run
Output
while i < len(a) and a[i] <= a[pivot]:
i += 1
while a[j] > a[pivot]:
j -= 1
if i < j:
a[i], a[j] = a[j], a[i]
a[j], a[pivot] = a[pivot], a[j]
quicksort(a, low, j - 1)
quicksort(a, j + 1, high)
a = list(map(int, input("Enter numbers to sort: ").split()))
n = len(a)
quicksort(a, 0, n - 1)
print("Sorted array:")
print(a)
Enter numbers to sort: 1 2 3 4 5
Sorted array:
[1, 2, 3, 4, 5]

PROGRAM-4(B):

AIM: To write a Python program to implement merge sort.

CODE:

```
def merging(arr, low, mid, high):
    result = []

    i = low
    j = mid + 1

    while i <= mid and j <= high:
        if arr[i] < arr[j]:
            result.append(arr[i])
            i += 1
        else:
            result.append(arr[j])
            j += 1

    while i <= mid:
        result.append(arr[i])
        i += 1

    while j <= high:
        result.append(arr[j])
        j += 1

    i = low
    j = 0

    while i <= high:
        arr[i] = result[j]
        i += 1
        j += 1


def merge_sort(arr, low, high):
    if low < high:
        mid = (low + high) // 2

        merge_sort(arr, low, mid)
        merge_sort(arr, mid + 1, high)

        merging(arr, low, mid, high)

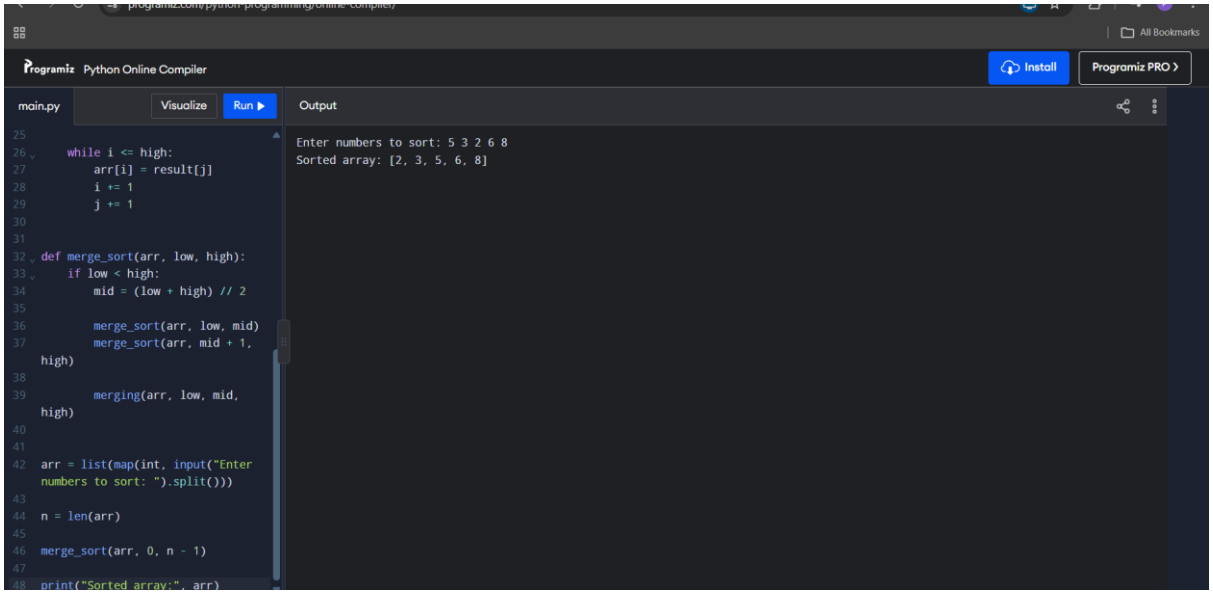

arr = list(map(int, input("Enter numbers to sort: ").split()))

n = len(arr)
```

```
merge_sort(arr, 0, n - 1)

print("Sorted array:", arr)
```

OUTPUT:



WEEK – 5

PROGRAM-5(A):

AIM: To write a Python program to implement a Singly Linked List with the operations: Create, Insert at Beginning, Insert at End, Insert at Index, Delete by Value, Delete First Node, Delete Last Node, Count number of nodes, Sort, Search, Display/Traverse and Exit.

CODE:

Part 1: Create, Insert, Delete, Count, Display

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def Create(self):
        n=int(input("Enter the number of Elements: "))
        for i in range(n):
            x=int(input("Enter a value: "))
            new=Node(x)
            if self.head is None:
                self.head=new
            else:
                temp=self.head
                while temp.next:
                    temp=temp.next
                temp.next=new

    def display(self):
        if self.head is None:
            print("List is empty")
        else:
            temp=self.head
            while temp:
                print(temp.data,end="--> ")
                temp=temp.next
            print(None)

    def insert_at_beg(self,data):
        new=Node(data)
        new.next=self.head
        self.head=new

    def insert_at_end(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
```

```
        else:
            temp=self.head
            while temp.next:
                temp=temp.next
            temp.next=new

    def insert_index(self, index, data):
        if index==0:
            self.insert_at_beg(data)
            return
        new=Node(data)
        temp=self.head
        for i in range(index-1):
            if temp.next is None:
                print("Invalid Index")
                return
            temp=temp.next
        new.next=temp.next
        temp.next=new

    def deleteAtBeg(self):
        if self.head is None:
            print("List is empty")
        else:
            temp=self.head
            self.head=temp.next
            print("Deleted Value is :",temp.data)

    def deleteAtEnd(self):
        if self.head is None:
            print("List is empty")
        elif self.head.next is None:
            self.head=None
        else:
            temp=self.head
            temp1=temp
            while temp.next:
                temp1=temp
                temp=temp.next
            temp1.next=None
# Delete specific value from the list
    def delete(self,value):
        if self.head is None:
            print("List is empty")
        else:
            temp=self.head
            if temp and temp.data==value:
                self.head=temp.next
                print("Value Deleted")
                return
            while temp and temp.data!=value:
```

```
                temp=temp.next
            if temp is None:
                print("Value not found")
            else:
                temp.next=temp.next.next
                print("Value Deleted")

    def  count(self):
        count=0
        temp=self.head
        while temp:
            count+=1
            temp=temp.next
        print("Number of nodes in the list:",count)


S1= SinglyLinkedList()
while True:
    #Menu using match case
    print("1. Create a Singly Linked List")
    print("2. Display the List")
    print("3. Insert at Beginning")
    print("4. Insert at End")
    print("5. Insert at Specific Index")
    print("6. Delete from Beginning")
    print("7. Delete from End")
    print("8. Delete Specific Value")
    print("9. Count Nodes")
    print("10. Exit")
    choice=int(input("Enter your choice: "))
    match choice:
        case 1:
            S1.Create()
        case 2:
            S1.display()
        case 3:
            data=int(input("Enter a value to insert at beginning: "))
            S1.insert_at_beg(data)
        case 4:
            data=int(input("Enter a value to insert at end: "))
            S1.insert_at_end(data)
        case 5:
            index=int(input("Enter the index to insert: "))
            data=int(input("Enter a value to insert: "))
            S1.insert_index(index,data)
        case 6:
            S1.deleteAtBeg()
        case 7:
            S1.deleteAtEnd()
        case 8:
            value=int(input("Enter the value to delete: "))
```

```
            S1.delete(value)
        case 9:
            S1.count()
        case 10:
            break
        case _:
            print("Invalid Choice")
```

Part 2: Search and Sort

```
class Node:
    def __init__(self,data):
        self.data=data
        self.next=None

class SinglyLinkedList:
    def __init__(self):
        self.head=None

    def Create(self):
        n=int(input("Enter number of elements: "))
        for i in range(n):
            x=int(input("Enter a value"))
            new=Node(x)
            if self.head is None:
                self.head=new
            else:
                temp=self.head
                while temp.next:
                    temp=temp.next
                temp.next=new
#k=target
    def search(self,k):
        if self.head is None:
            print("Cannot perform search")
            return
        temp=self.head
        while temp:
            if temp.data==k:
                print(f"{k} is found ")
                return
            temp=temp.next
        print(f"{k} not found")

    def display(self):
        if self.head is None:
            print("No data")
            return
        temp=self.head
        while temp:
            print(temp.data,end='-->')
            temp=temp.next
```

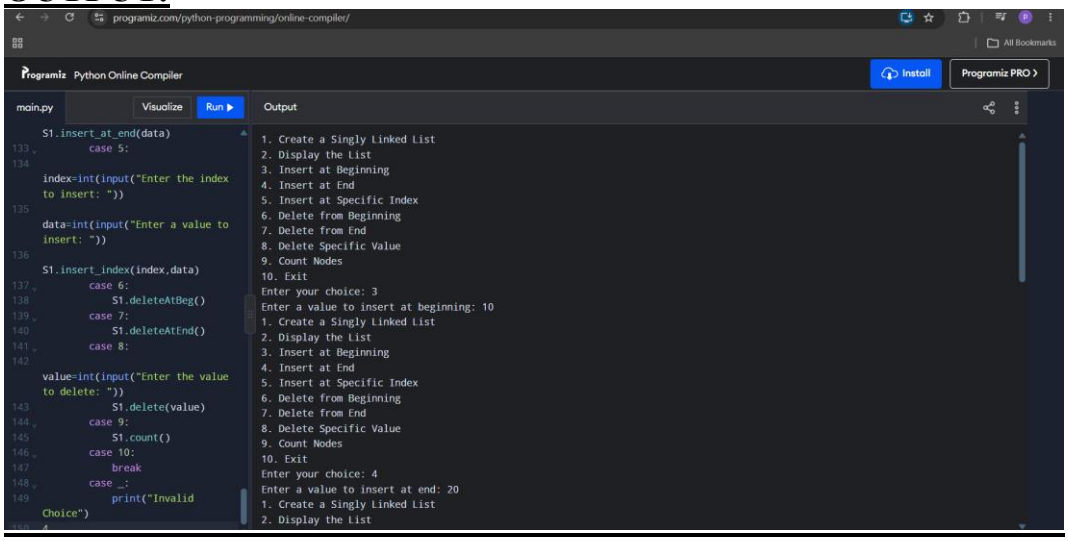
```
        print("None")

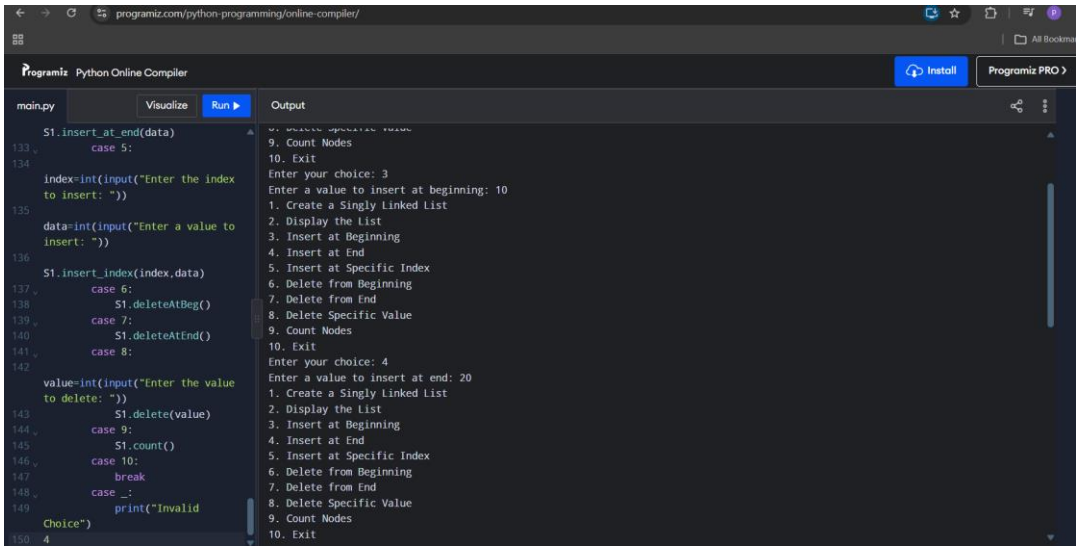
    def sort(self):
        if self.head is None:
            print("Cannot perform sorting")
            return
        i=self.head
        while i:
            j=i.next
            while j:
                if i.data>j.data:
                    i.data,j.data=j.data,i.data
                j=j.next
            i=i.next
        print("Sorting Completed")

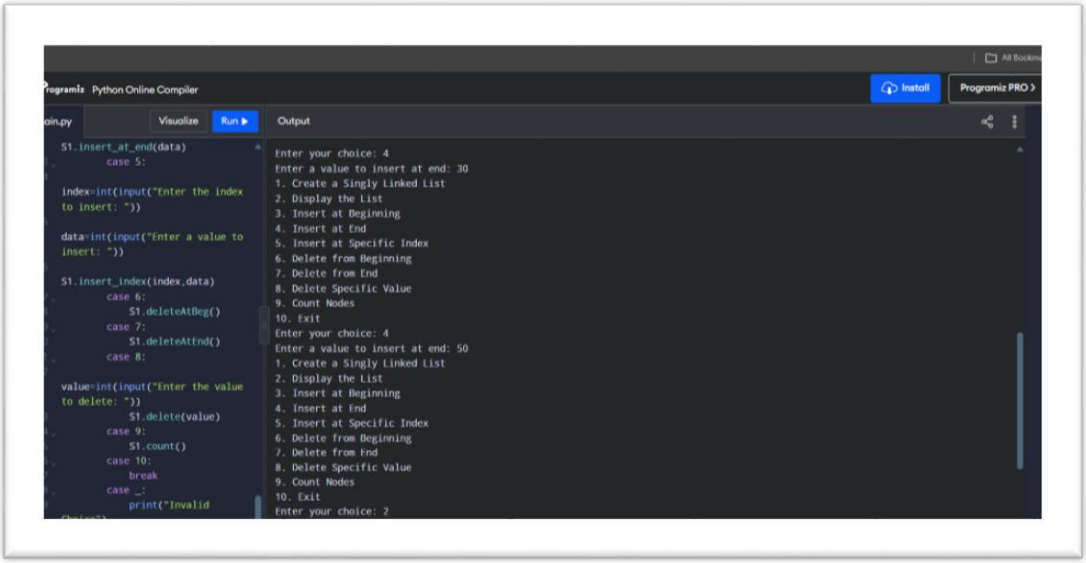
s11=SinglyLinkedList()

while True:
    print('1.Create a Linkedlist')
    print('2.Display')
    print('3.Search ')
    print('4.Sort')
    print('5.exit')
    n=int(input("Enter your choice: "))
    match(n):
        case 1:
            s11.Create()
        case 2:
            s11.display()
        case 3:
            n=int(input("Enter your target: "))
            s11.search(n)
        case 4:
            s11.sort()
        case 5:
            break;
        case _:
            print("Invalid Choice")
```

OUTPUT:


programiz.com/python-programming/online-compiler/
Programiz Python Online Compiler
Install
Programiz PRO
main.py
Visualize
Run
Output
S1.insert_at_end(data)
case 5:
index=int(input("Enter the index to insert: "))
data=int(input("Enter a value to insert: "))
S1.insert_index(index,data)
case 6:
S1.deleteAtBeg()
case 7:
S1.deleteAtEnd()
case 8:
value=int(input("Enter the value to delete: "))
S1.delete(value)
case 9:
S1.count()
case 10:
break
case _:
print("Invalid Choice")
1. Create a Singly Linked List
2. Display the List
3. Insert at Beginning
4. Insert at End
5. Insert at Specific Index
6. Delete from Beginning
7. Delete from End
8. Delete Specific Value
9. Count Nodes
10. Exit
Enter your choice: 3
Enter a value to insert at beginning: 10
1. Create a Singly Linked List
2. Display the List
3. Insert at Beginning
4. Insert at End
5. Insert at Specific Index
6. Delete from Beginning
7. Delete from End
8. Delete Specific Value
9. Count Nodes
10. Exit
Enter your choice: 4
Enter a value to insert at end: 20
1. Create a Singly Linked List
2. Display the List


programiz.com/python-programming/online-compiler/
Programiz Python Online Compiler
Install
Programiz PRO
main.py
Visualize
Run
Output
S1.insert_at_end(data)
case 5:
index=int(input("Enter the index to insert: "))
data=int(input("Enter a value to insert: "))
S1.insert_index(index,data)
case 6:
S1.deleteAtBeg()
case 7:
S1.deleteAtEnd()
case 8:
value=int(input("Enter the value to delete: "))
S1.delete(value)
case 9:
S1.count()
case 10:
break
case _:
print("Invalid Choice")
9. Count Nodes
10. Exit
Enter your choice: 3
Enter a value to insert at beginning: 10
1. Create a Singly Linked List
2. Display the List
3. Insert at Beginning
4. Insert at End
5. Insert at Specific Index
6. Delete from Beginning
7. Delete from End
8. Delete Specific Value
9. Count Nodes
10. Exit
Enter your choice: 4
Enter a value to insert at end: 20
1. Create a Singly Linked List
2. Display the List
3. Insert at Beginning
4. Insert at End
5. Insert at Specific Index
6. Delete from Beginning
7. Delete from End
8. Delete Specific Value
9. Count Nodes
10. Exit


Programiz Python Online Compiler
Install
Programiz PRO
Visualize
Run
Output
S1.insert_at_end(data)
case 5:
index=int(input("Enter the index to insert: "))
data=int(input("Enter a value to insert: "))
S1.insert_index(index,data)
case 6:
S1.deleteAtBeg()
case 7:
S1.deleteAtEnd()
case 8:
value=int(input("Enter the value to delete: "))
S1.delete(value)
case 9:
S1.count()
case 10:
break
case _:
print("Invalid
Enter your choice: 4
Enter a value to insert at end: 30
1. Create a Singly Linked List
2. Display the List
3. Insert at Beginning
4. Insert at End
5. Insert at Specific Index
6. Delete from Beginning
7. Delete from End
8. Delete Specific Value
9. Count Nodes
10. Exit
Enter your choice: 4
Enter a value to insert at end: 50
1. Create a Singly Linked List
2. Display the List
3. Insert at Beginning
4. Insert at End
5. Insert at Specific Index
6. Delete from Beginning
7. Delete from End
8. Delete Specific Value
9. Count Nodes
10. Exit
Enter your choice: 2

WEEK – 6

PROGRAM-6:

AIM: To write a Python program to implement a Doubly Linked List with the operations: Create, Insert at Beginning, Insert at End, Insert at Index, Delete by Value, Delete First Node, Delete Last Node, Count number of nodes, Display/Traverse and Exit.

CODE:

```
class Node:
    def __init__(self, data):
        self.prev = None
        self.data = data
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # 1. Insert at Beginning
    def insert_begin(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            new.next = self.head
            self.head.prev = new
            self.head = new

        print("Insertion Completed")

    # 2. Insert at End
    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head

            while temp.next:
                temp = temp.next

            temp.next = new
```

```
            new.prev = temp

        print("Insertion Completed")

    # 3. Insert at Specific Position (Index)
    def insert_index(self, index, data):
        if index < 0:
            print("Invalid Index")

        elif index == 0:
            self.insert_begin(data)
            return

        else:
            new = Node(data)
            temp = self.head

            for i in range(index - 1):
                if temp is None:
                    print("Invalid Index")
                    return
                temp = temp.next

            if temp is None:
                print("Invalid Index")
                return

            new.next = temp.next
            new.prev = temp

            if temp.next:
                temp.next.prev = new

            temp.next = new

            print("Insertion Completed")

    # 4. Delete at Beginning
    def delete_begin(self):
        if self.head is None:
            print("Can't perform delete operation")

        elif self.head.next is None:
            self.head = None
            print("Value Deleted")

        else:
            self.head.next.prev = None
```

```
            self.head = self.head.next
            print("Value Deleted")

    # 5. Delete at End
    def delete_end(self):
        if self.head is None:
            print("Can't perform delete operation")

        elif self.head.next is None:
            self.head = None
            print("Value Deleted")

        else:
            temp = self.head

            while temp.next:
                temp = temp.next

            temp.prev.next = None
            temp.prev = None

            print("Value Deleted")

    # 6. Delete at Specific Position (Index)
    def delete_index(self, index):
        if self.head is None:
            print("Can't perform delete operation")

        elif index < 0:
            print("Invalid Index")

        elif index == 0:
            self.delete_begin()

        else:
            temp = self.head

            for i in range(index):
                if temp is None:
                    print("Invalid Index")
                    return
                temp = temp.next

            if temp is None:
                print("Invalid Index")

            elif temp.next:
                temp.next.prev = temp.prev
```

```
                temp.prev.next = temp.next
                print("Value Deleted")

            else:
                temp.prev.next = None
                temp.prev = None
                print("Value Deleted")

    # 7. Forward Display
    def display_fwd(self):
        temp = self.head

        if temp is None:
            print("List is Empty")
            return

        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")

    # 8. Backward Display
    def display_bwd(self):
        if self.head is None:
            print("List is Empty")
            return

        temp = self.head

        while temp.next:
            temp = temp.next

        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.prev

        print("None")


# Main Menu
dll = DoublyLinkedList()

while True:
    print("\n----- DOUBLY LINKED LIST MENU -----")
    print("1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Insert at Specific Position")
    print("4. Delete at Beginning")
```

```
    print("5. Delete at End")
    print("6. Delete at Specific Position")
    print("7. Display Forward")
    print("8. Display Backward")
    print("9. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        dll.insert_begin(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        dll.insert_end(data)

    elif choice == 3:
        pos = int(input("Enter index: "))
        data = int(input("Enter data: "))
        dll.insert_index(pos, data)

    elif choice == 4:
        dll.delete_begin()

    elif choice == 5:
        dll.delete_end()

    elif choice == 6:
        pos = int(input("Enter index: "))
        dll.delete_index(pos)

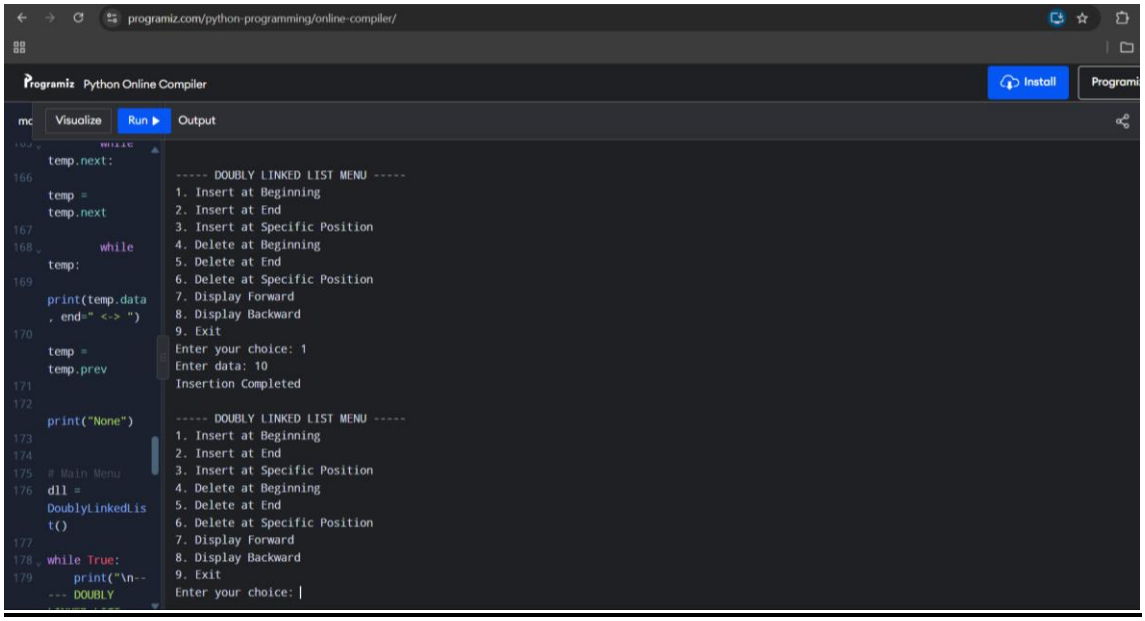
    elif choice == 7:
        dll.display_fwd()

    elif choice == 8:
        dll.display_bwd()

    elif choice == 9:
        print("Exiting Program")
        break

    else:
        print("Invalid Choice")
```

OUTPUT:


programiz.com/python-programming/online-compiler/
Programiz Python Online Compiler
Install
Visualize
Run
Output
temp.next:
temp = temp.next
while temp:
print(temp.data, end=" <-> ")
temp = temp.prev
print("None")
Main Menu
dll = DoublyLinkedList()
while True:
print("\n--- DOUBLY
----- DOUBLY LINKED LIST MENU -----
1. Insert at Beginning
2. Insert at End
3. Insert at Specific Position
4. Delete at Beginning
5. Delete at End
6. Delete at Specific Position
7. Display Forward
8. Display Backward
9. Exit
Enter your choice: 1
Enter data: 10
Insertion Completed
----- DOUBLY LINKED LIST MENU -----
1. Insert at Beginning
2. Insert at End
3. Insert at Specific Position
4. Delete at Beginning
5. Delete at End
6. Delete at Specific Position
7. Display Forward
8. Display Backward
9. Exit
Enter your choice:

WEEK – 7

PROGRAM-7(A):

AIM: To write a Python program to implement stack operations such as push, pop, peek and display using an array.

CODE:

```
class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    # PUSH
    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "inserted")

    # POP
    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.top -= 1
            print("Deleted value =", item)

    # PEEK
    def peek(self):
        if self.top == -1:
            print("Stack is Empty")
        else:
            print("Top element =", self.stack[self.top])

    # DISPLAY
    def display(self):
        if self.top == -1:
            print("Stack is Empty")
        else:
            print("Stack elements:")
            for i in range(self.top, -1, -1):
                print(self.stack[i])
```

```
# Main Program
size = int(input("Enter stack size: "))
s = Stack(size)

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        item = int(input("Enter element: "))
        s.push(item)

    elif choice == 2:
        s.pop()

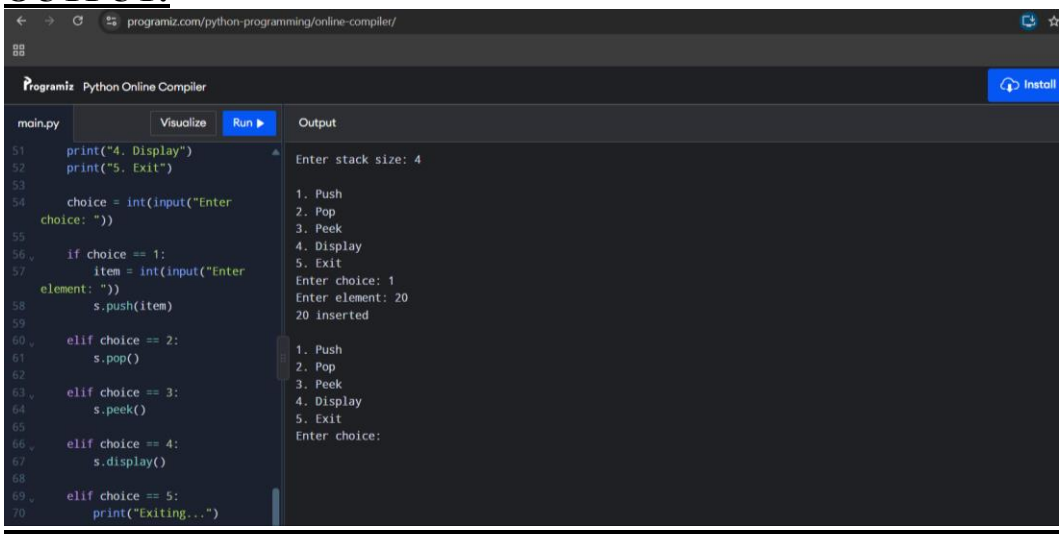
    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Exiting...")
        break

    else:
        print("Invalid Choice")
```

OUTPUT:


programiz.com/python-programming/online-compiler/
Programiz Python Online Compiler
Install
main.py
Visualize
Run
Output
print("4. Display")
print("5. Exit")
choice = int(input("Enter choice: "))
if choice == 1:
item = int(input("Enter element: "))
s.push(item)
elif choice == 2:
s.pop()
elif choice == 3:
s.peek()
elif choice == 4:
s.display()
elif choice == 5:
print("Exiting...")
Enter stack size: 4
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter element: 20
20 inserted
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice:

PROGRAM-7(B):

AIM: To write a Python program to implement stack operations such as push, pop, peek and display using a linked list.

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    # PUSH
    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new
        print(data, "inserted")

    # POP
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp = self.top
            print(temp.data, "popped from stack")
            self.top = self.top.next

    # PEEK
    def peek(self):
        if self.top is None:
            print("Stack is Empty")
        else:
            print("Top element =", self.top.data)

    # DISPLAY
    def display(self):
        if self.top is None:
            print("Stack is Empty")
        else:
            temp = self.top
            print("Stack elements:")
```

```
            while temp:
                print(temp.data)
                temp = temp.next


# Main Program
s = Stack()

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        data = int(input("Enter element: "))
        s.push(data)

    elif choice == 2:
        s.pop()

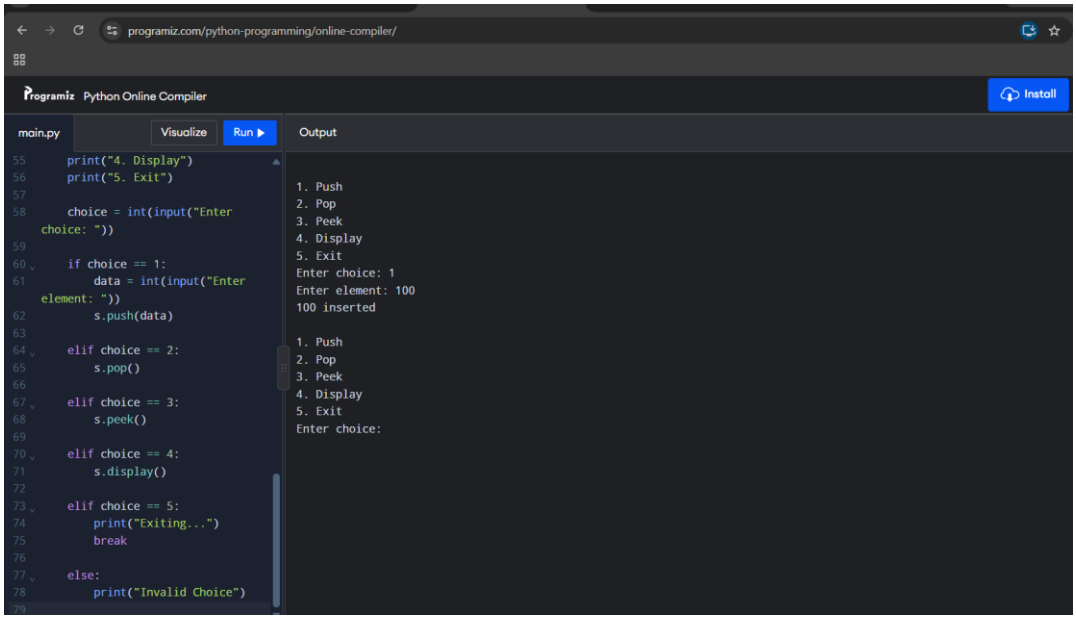
    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Exiting...")
        break

    else:
        print("Invalid Choice")
```

OUTPUT:


programiz.com/python-programming/online-compiler/
Programiz Python Online Compiler
Install
main.py
Visualize
Run
Output
print("4. Display")
print("5. Exit")
choice = int(input("Enter choice: "))
if choice == 1:
data = int(input("Enter element: "))
s.push(data)
elif choice == 2:
s.pop()
elif choice == 3:
s.peek()
elif choice == 4:
s.display()
elif choice == 5:
print("Exiting...")
break
else:
print("Invalid Choice")
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter element: 100
100 inserted
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice:

WEEK – 8

PROGRAM-8(A):

AIM: To write a Python program to implement a Queue and its operations such as enqueue, dequeue, peek and display with a menu driven approach, using array and linked list data structures.

CODE:

Queue using array

```
class Queue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    # ENQUEUE
    def enqueue(self, data):
        if self.rear == self.size - 1:
            print("Queue Overflow")

        elif self.front == -1 and self.rear == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = data
            print(data, "inserted")

        else:
            self.rear += 1
            self.queue[self.rear] = data
            print(data, "inserted")

    # DEQUEUE
    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")

        else:
            item = self.queue[self.front]
            print("Deleted element =", item)
            self.front += 1

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    # PEEK
```

```
    def peek(self):
        if self.front == -1:
            print("Queue is Empty")
        else:
            print("Front element =", self.queue[self.front])

    # DISPLAY
    def display(self):
        if self.front == -1:
            print("Queue is Empty")
        else:
            print("Queue elements:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i], end=" ")
            print()


# Main Program
size = int(input("Enter queue size: "))
q = Queue(size)

while True:
    print("\n----- QUEUE USING ARRAY -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        data = int(input("Enter element: "))
        q.enqueue(data)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting...")
        break

    else:
```

```
        print("Invalid Choice")
```

Queue using linked list

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    # ENQUEUE
    def enqueue(self, data):
        new = Node(data)

        if self.front is None:
            self.front = new
            self.rear = new

        else:
            self.rear.next = new
            self.rear = new

        print(data, "inserted")

    # DEQUEUE
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")

        else:
            temp = self.front
            print("Deleted element =", temp.data)

            self.front = self.front.next

            if self.front is None:
                self.rear = None

    # PEEK
    def peek(self):
        if self.front is None:
            print("Queue is Empty")
```

```
        else:
            print("Front element =", self.front.data)

    # DISPLAY
    def display(self):
        if self.front is None:
            print("Queue is Empty")

        else:
            temp = self.front

            print("Queue elements:")

            while temp:
                print(temp.data, end=" -> ")
                temp = temp.next

            print("None")


# Main Program
q = Queue()

while True:
    print("\n----- QUEUE USING LINKED LIST -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        data = int(input("Enter element: "))
        q.enqueue(data)

    elif choice == 2:
        q.dequeue()

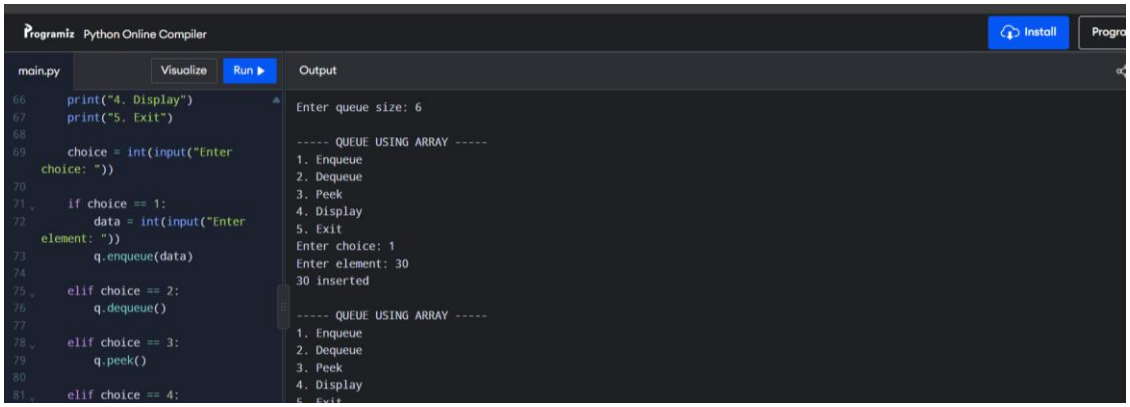
    elif choice == 3:
        q.peek()

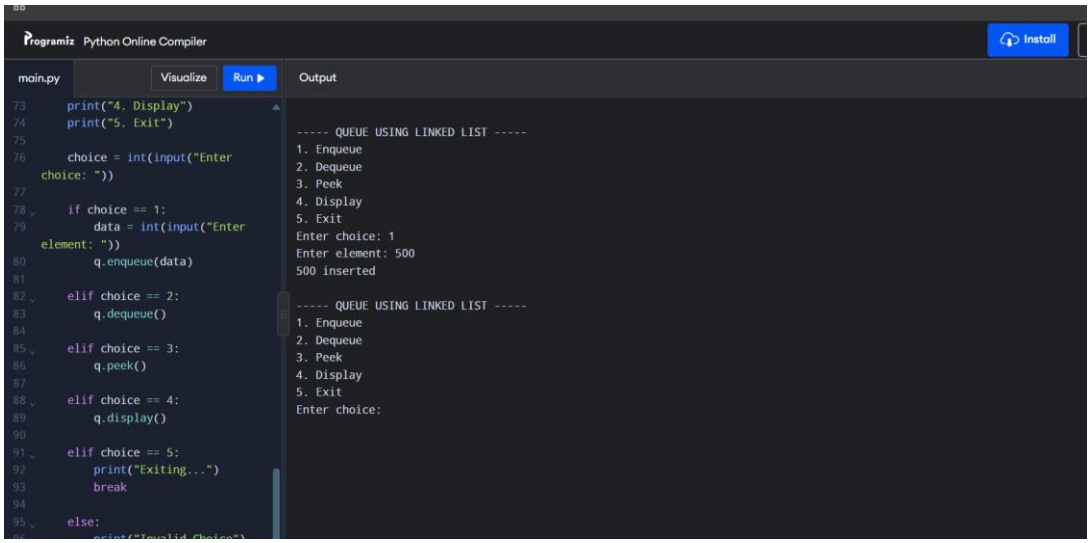
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting...")
        break
```

```
    else:
        print("Invalid Choice")
```

OUTPUT:





PROGRAM-8(B):

AIM: To write a Python program to implement a Circular Queue and its operations such as enqueue, dequeue, peek and display with a menu driven approach, using array and linked list data structures.

CODE:

Circular Queue using array

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into the queue")


    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        # Only one element is present
        elif self.front == self.rear:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = -1
            self.rear = -1
            print(item, "deleted from the queue")
        # More than one element
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = (self.front + 1) % self.size
            print(item, "deleted from the queue")

    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
```

```
            print("The elements of the queue are:")
            i = self.front
            while True:
                print(self.queue[i])
                if i == self.rear:
                    break
                i = (i + 1) % self.size

# Create Circular Queue
size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)
while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.display()
    elif choice == 4:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

Circular Queue using linked list

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    # ENQUEUE
    def enqueue(self, data):
        new = Node(data)

        if self.front is None:
            self.front = new
```

```
            self.rear = new
            new.next = self.front

        else:
            self.rear.next = new
            self.rear = new
            self.rear.next = self.front

        print(data, "inserted")

    # DEQUEUE
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")

        elif self.front == self.rear:
            print("Deleted element =", self.front.data)
            self.front = None
            self.rear = None

        else:
            temp = self.front
            print("Deleted element =", temp.data)

            self.front = self.front.next
            self.rear.next = self.front

    # PEEK
    def peek(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            print("Front element =", self.front.data)

    # DISPLAY
    def display(self):
        if self.front is None:
            print("Queue is Empty")

        else:
            temp = self.front

            print("Queue elements:")

            while True:
                print(temp.data, end=" -> ")
                temp = temp.next
```

```
                if temp == self.front:
                    break

            print("(Back to Front)")


# Main Program
q = CircularQueue()

while True:
    print("\n----- CIRCULAR QUEUE USING LL -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        data = int(input("Enter element: "))
        q.enqueue(data)

    elif choice == 2:
        q.dequeue()

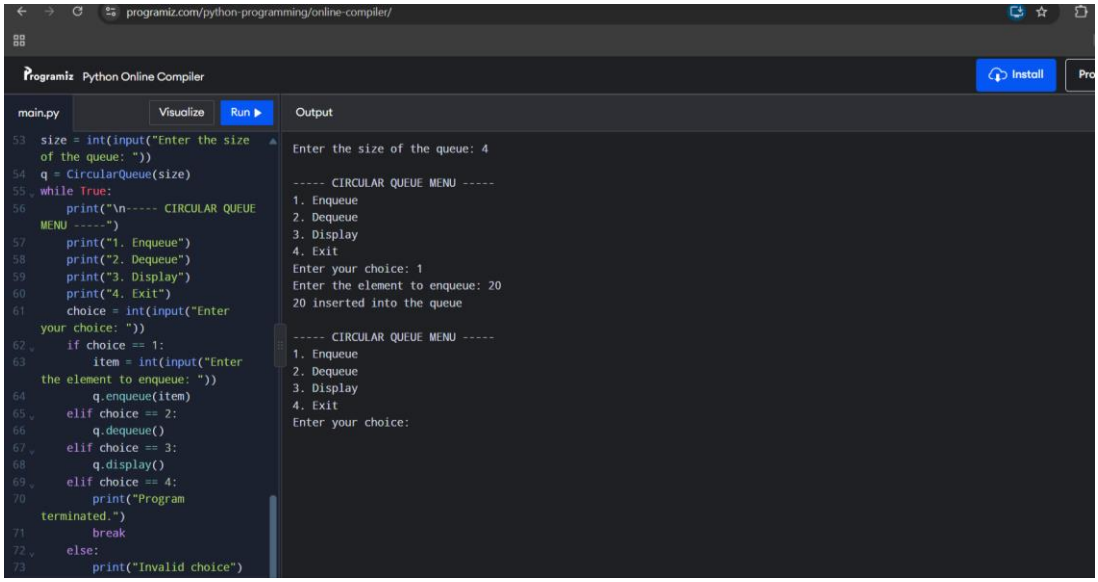
    elif choice == 3:
        q.peek()

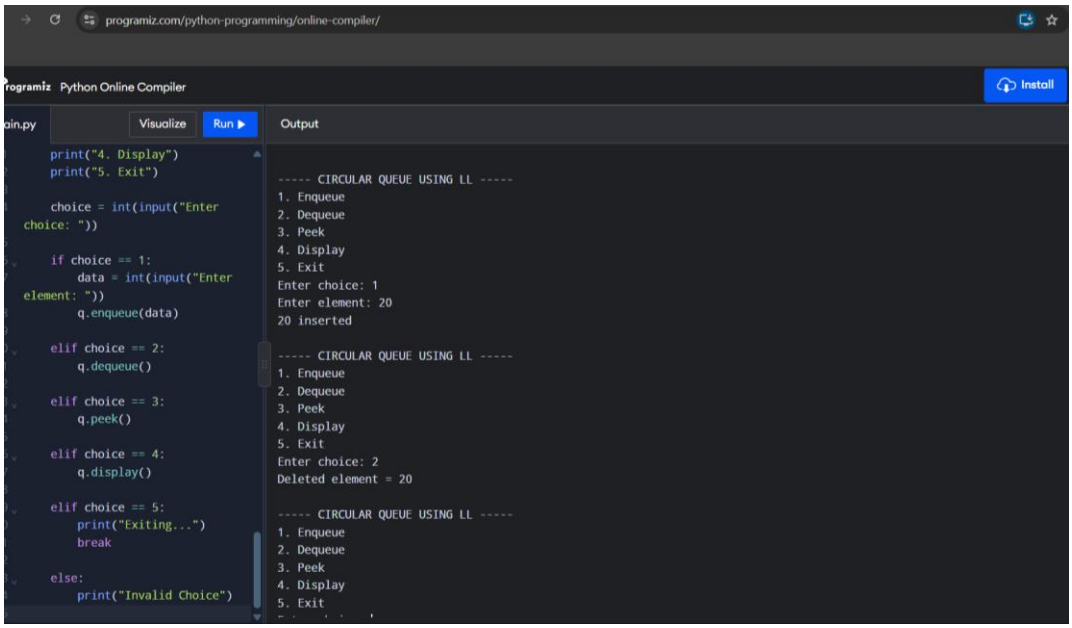
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting...")
        break

    else:
        print("Invalid Choice")
```

OUTPUT:


programiz.com/python-programming/online-compiler/
Programiz Python Online Compiler
Install
main.py
Visualize
Run
Output
size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)
while True:
print("\n----- CIRCULAR QUEUE MENU -----")
print("1. Enqueue")
print("2. Dequeue")
print("3. Display")
print("4. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
item = int(input("Enter the element to enqueue: "))
q.enqueue(item)
elif choice == 2:
q.dequeue()
elif choice == 3:
q.display()
elif choice == 4:
print("Program terminated.")
break
else:
print("Invalid choice")
Enter the size of the queue: 4
----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the element to enqueue: 20
20 inserted into the queue
----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice:


programiz.com/python-programming/online-compiler/
Python Online Compiler
Install
Visualize
Run
Output
print("4. Display")
print("5. Exit")
choice = int(input("Enter choice: "))
if choice == 1:
data = int(input("Enter element: "))
q.enqueue(data)
elif choice == 2:
q.dequeue()
elif choice == 3:
q.peek()
elif choice == 4:
q.display()
elif choice == 5:
print("Exiting...")
break
else:
print("Invalid Choice")
----- CIRCULAR QUEUE USING LL -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter element: 20
20 inserted
----- CIRCULAR QUEUE USING LL -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 2
Deleted element = 20
----- CIRCULAR QUEUE USING LL -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

WEEK – 9

PROGRAM-9:

AIM: To write a Python program to implement a Binary Search Tree (BST) and perform Insertion, Deletion, Tree Traversals (in-order, pre-order, post-order), Count Leaf Nodes, Count Internal Nodes and find the Height of the tree.

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")

def find_min(root):
    if root is None:
```

```
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1


def delete(root, x):
    if root is None:
```

```
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
            root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)


def level_order(root):
    h = height(root)
    for level in range(h):
        print_level(root, level)


# ====================================================
# MAIN PROGRAM
# ====================================================

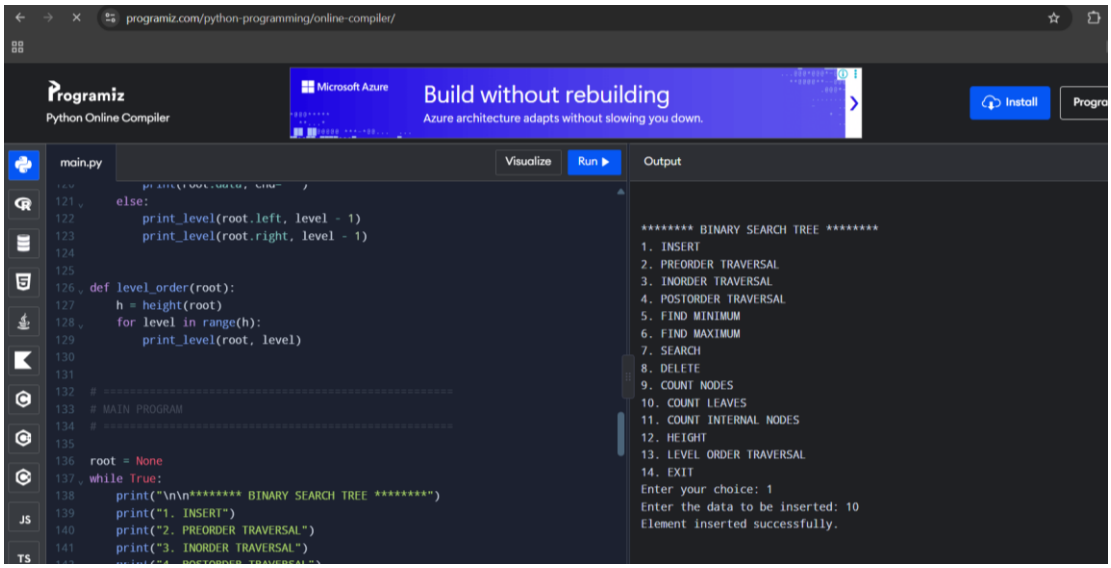
root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
```

```
    print("6. FIND MAXIMUM")
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
    print("10. COUNT LEAVES")
    print("11. COUNT INTERNAL NODES")
    print("12. HEIGHT")
    print("13. LEVEL ORDER TRAVERSAL")
    print("14. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            preorder(root)
            print()
    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            inorder(root)
            print()
    elif choice == 4:
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            postorder(root)
            print()
    elif choice == 5:
        node = find_min(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
```

```
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")
    elif choice == 8:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 9:
        print("Number of nodes:", count_nodes(root))
    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 12:
        print("Height of BST:", height(root))
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()
    elif choice == 14:
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:



WEEK – 10

PROGRAM-10:

AIM: To write a menu driven Python program to implement an AVL Tree and perform Insertion, Deletion, Tree Traversals, Count Leaf Nodes, Count Internal Nodes and find the Height of the tree.

CODE:

```
class Node:
    def __init__(self, x):
        self.left = None
        self.data = x
        self.right = None
        self.h = 0

#AVL TREE Program
def height(root):
    if root is None:
        return -1
    else:
        return root.h

# LL Rotation
def LL(x):
    y = x.left
    x.left = y.right
    y.right = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# RR Rotation
def RR(x):
    y = x.right
    x.right = y.left
    y.left = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# LR Rotation
def LR(k3):
    k3.left = RR(k3.left)
    return LL(k3)

# RL Rotation
def RL(k3):
    k3.right = LL(k3.right)
    return RR(k3)
```

```
# Inorder Traversal
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


# Preorder Traversal
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)


# Postorder Traversal
def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

# Insert a node into AVL Tree
def insert(root, x):
    if root is None:
        return Node(x)

    if x < root.data:
        root.left = insert(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    elif x > root.data:
        root.right = insert(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
    else:
        print("Duplicates not allowed")
    root.h = max(height(root.left),height(root.right)) + 1
    return root


# Delete a node from AVL Tree
```

```
def delete(root, x):
    # Element not found
    if root is None:
        return None

    # Search in left subtree
    if x < root.data:
        root.left = delete(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    # Search in right subtree
    elif x > root.data:
        root.right = delete(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)

    # When target found
    else:
        if root.left is None and root.right is None:
            root = None
        elif root.right is None:
            root = root.left
        elif root.left is None:
            root = root.right
        else:
            successor = root.right   # Find inorder successor
            while successor.left is not None:
                successor = successor.left
            root.data = successor.data   #Copy successor data
            root.right = delete(root.right, successor.data)     # Delete the
inorder successor

        root.h = max(height(root.left), height(root.right)) + 1
    return root

# ------------------------------
# Main Program
# ------------------------------

root = None

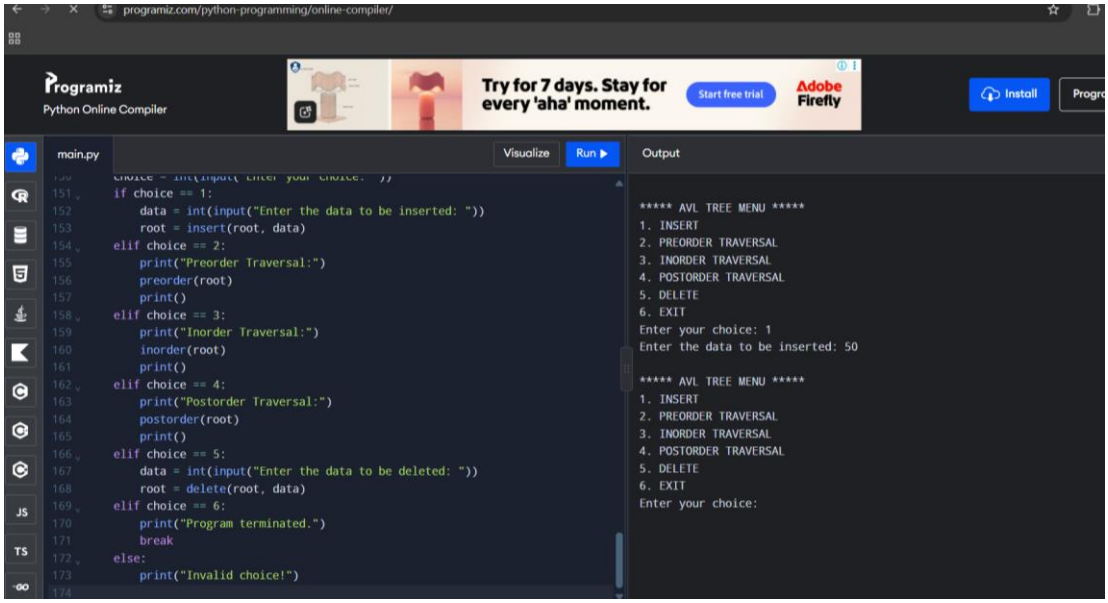
while True:

    print("\n***** AVL TREE MENU *****")
```

```
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. DELETE")
    print("6. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
    elif choice == 2:
        print("Preorder Traversal:")
        preorder(root)
        print()
    elif choice == 3:
        print("Inorder Traversal:")
        inorder(root)
        print()
    elif choice == 4:
        print("Postorder Traversal:")
        postorder(root)
        print()
    elif choice == 5:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 6:
        print("Program terminated.")
        break
    else:
        print("Invalid choice!")
```

OUTPUT:


programiz.com/python-programming/online-compiler/
Programiz
Python Online Compiler
Try for 7 days. Stay for every 'aha' moment.
Start free trial
Adobe Firefly
Install
main.py
Visualize
Run
Output
if choice == 1:
data = int(input("Enter the data to be inserted: "))
root = insert(root, data)
elif choice == 2:
print("Preorder Traversal:")
preorder(root)
print()
elif choice == 3:
print("Inorder Traversal:")
inorder(root)
print()
elif choice == 4:
print("Postorder Traversal:")
postorder(root)
print()
elif choice == 5:
data = int(input("Enter the data to be deleted: "))
root = delete(root, data)
elif choice == 6:
print("Program terminated.")
break
else:
print("Invalid choice!")
***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 1
Enter the data to be inserted: 50
***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice:

WEEK – 11

PROGRAM-11:

AIM: To write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform Insertion, Deletion of the root element and Display of the array representation of the heap.

CODE:

Max Heap

```
def insert(heap, value):
    heap.append(value)
    i = len(heap) - 1
    while i > 0:
        parent = (i - 1) // 2
        if heap[i] > heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break

def delete(heap):
    if len(heap) == 0:
        print("Heap is empty")
        return
    print("Deleted element:", heap[0])
    heap[0] = heap[-1]
    heap.pop()

    n = len(heap)
    i = 0
    # Move the root downward
    while True:
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n and heap[left] > heap[largest]:
            largest = left
        if right < n and heap[right] > heap[largest]:
            largest = right
        if largest != i:
            heap[i], heap[largest] = heap[largest], heap[i]
            i = largest
        else:
            break

def display(heap):
    if len(heap) == 0:
        print("Heap is empty")
    else:
```

```
        print("Heap elements:", heap)

#Main Program
heap = []
while True:
    print("\n--- MAX HEAP MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        value = int(input("Enter value to insert: "))
        insert(heap, value)
    elif choice == 2:
        delete(heap)
    elif choice == 3:
        display(heap)
    elif choice == 4:
        print("Exiting program")
        break
    else:
        print("Invalid choice")
```

Min Heap

```
def insert(heap, value):
    heap.append(value)
    i = len(heap) - 1

    while i > 0:
        parent = (i - 1) // 2

        if heap[i] < heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break


def delete(heap):
    if len(heap) == 0:
        print("Heap is empty")
        return

    print("Deleted element:", heap[0])

    heap[0] = heap[-1]
    heap.pop()
```

```
    n = len(heap)
    i = 0

    # Move the root downward
    while True:
        smallest = i
        left = 2 * i + 1
        right = 2 * i + 2

        if left < n and heap[left] < heap[smallest]:
            smallest = left

        if right < n and heap[right] < heap[smallest]:
            smallest = right

        if smallest != i:
            heap[i], heap[smallest] = heap[smallest], heap[i]
            i = smallest
        else:
            break


def display(heap):
    if len(heap) == 0:
        print("Heap is empty")
    else:
        print("Heap elements:", heap)


# Main Program
heap = []

while True:
    print("\n--- MIN HEAP MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value to insert: "))
        insert(heap, value)

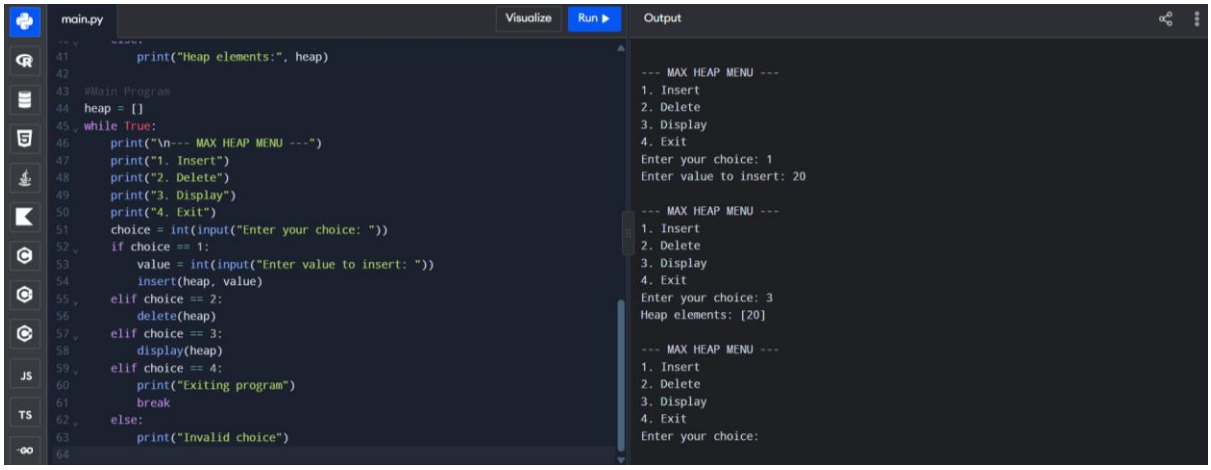
    elif choice == 2:
        delete(heap)
```

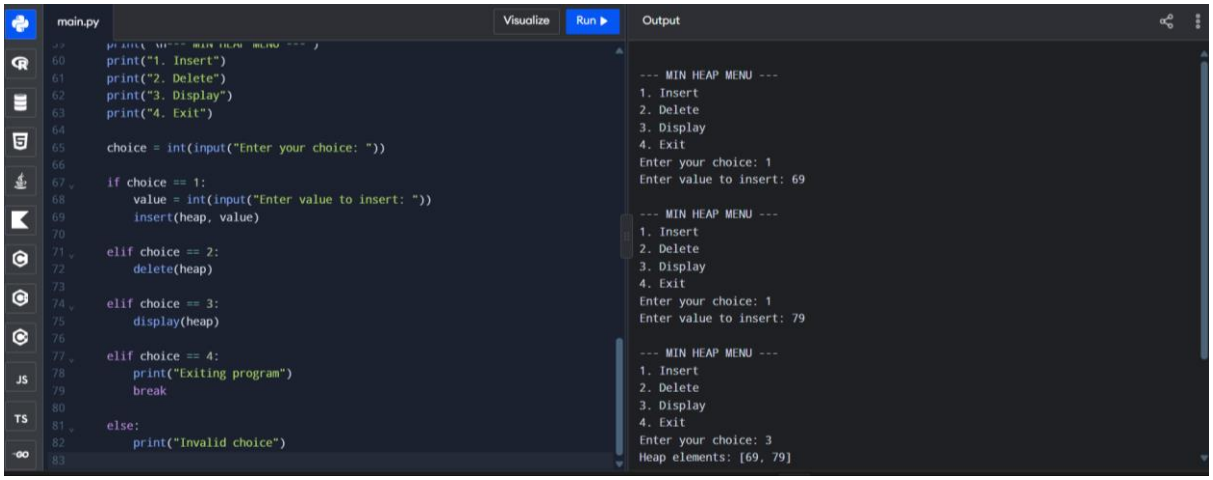
```
    elif choice == 3:
        display(heap)

    elif choice == 4:
        print("Exiting program")
        break

    else:
        print("Invalid choice")
```

OUTPUT:





WEEK – 12

PROGRAM-12(A):

AIM: To write a Python program to implement a Hash Table using Separate Chaining for collision resolution.

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

SIZE = 10
ht = [None] * SIZE

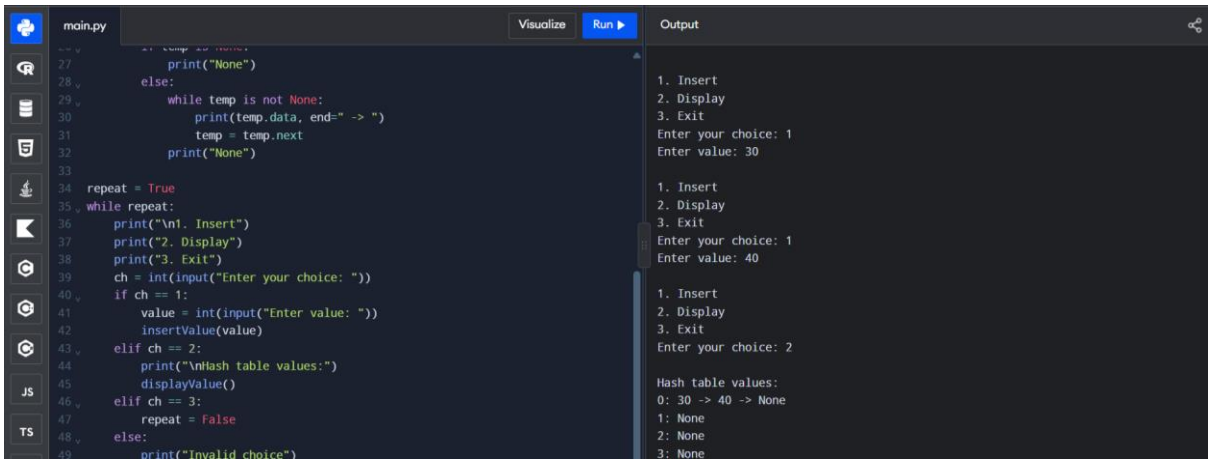

def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode

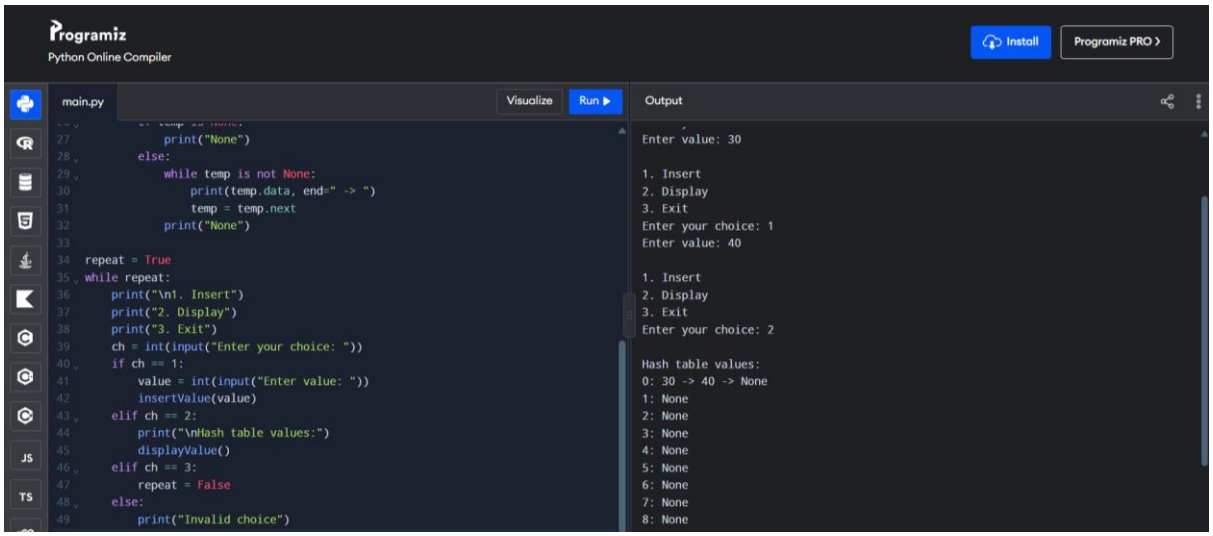

def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
```

```
    elif ch == 2:
        print("\nHash table values:")
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

OUTPUT:





PROGRAM-12(B):

AIM: To write a Python program to implement a Hash Table using Linear Probing for collision resolution.

CODE:

```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")


def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")

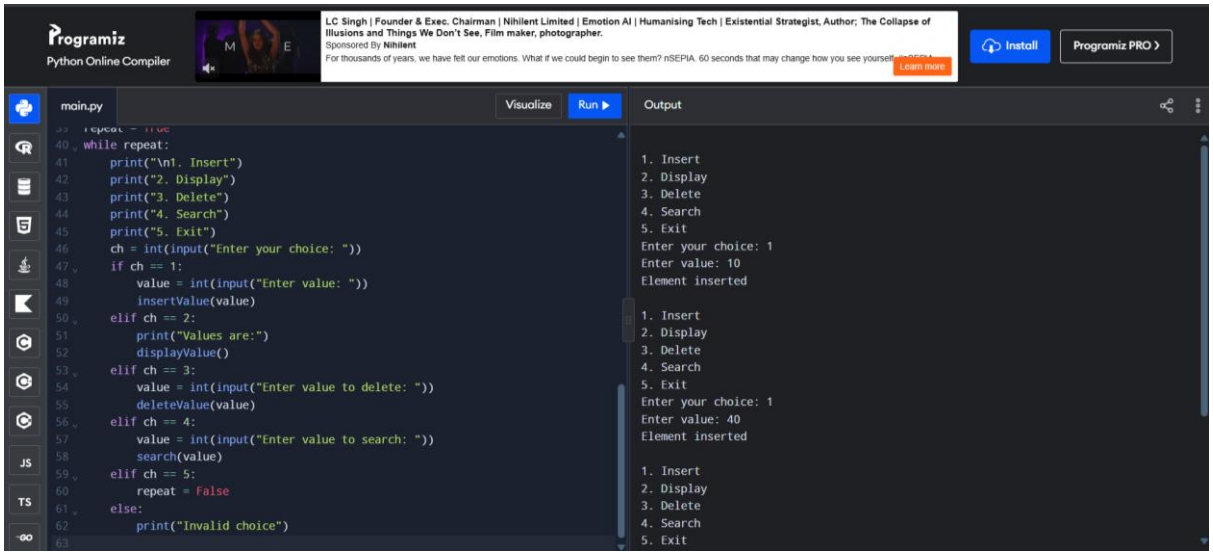
def search(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            print("Element found at index", index)
            return
        if a[index] == -1:
            break
    print("Element not found")

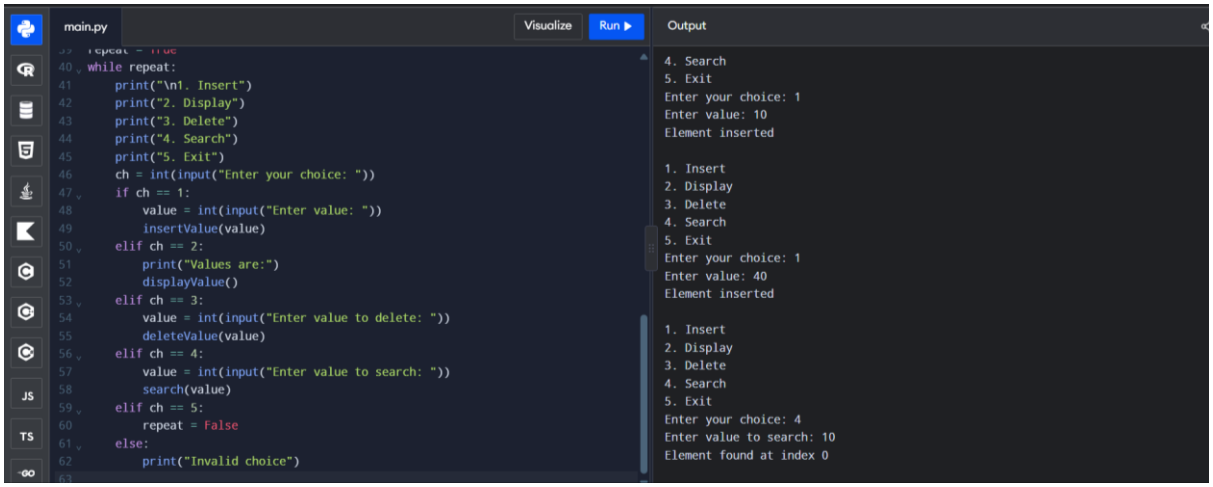

def displayValue():
    print("Index\tValue")
    for i in range(SIZE):
        print(i, "\t", a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Delete")
    print("4. Search")
    print("5. Exit")
```

```
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("Values are:")
        displayValue()
    elif ch == 3:
        value = int(input("Enter value to delete: "))
        deleteValue(value)
    elif ch == 4:
        value = int(input("Enter value to search: "))
        search(value)
    elif ch == 5:
        repeat = False
    else:
        print("Invalid choice")
```

OUTPUT:





PROGRAM-12(C):

AIM: To write a Python program to implement a Hash Table using Quadratic Probing for collision resolution.

CODE:

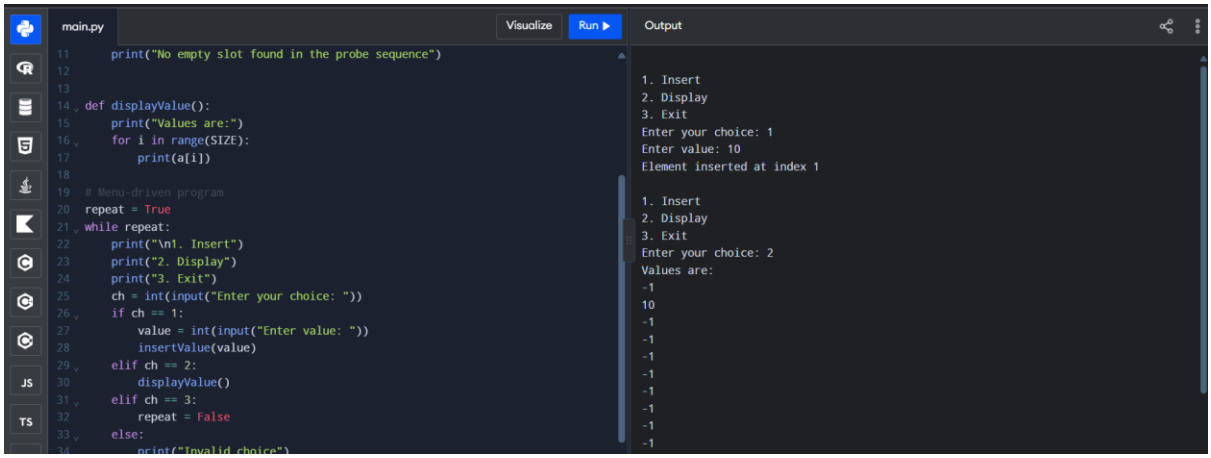
```
SIZE = 10
a = [-1] * SIZE

def insertValue(value):
    for i in range(SIZE):
        index = ((value % SIZE) + i * i) % SIZE
    if a[index] == -1:
            a[index] = value
            print("Element inserted at index", index)
            return
    print("No empty slot found in the probe sequence")


def displayValue():
    print("Values are:")
    for i in range(SIZE):
        print(a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

OUTPUT:


main.py
Visualize
Run
Output
print("No empty slot found in the probe sequence")
def displayValue():
print("Values are:")
for i in range(SIZE):
print(a[i])
Menu-driven program
repeat = True
while repeat:
print("\n1. Insert")
print("2. Display")
print("3. Exit")
ch = int(input("Enter your choice: "))
if ch == 1:
value = int(input("Enter value: "))
insertValue(value)
elif ch == 2:
displayValue()
elif ch == 3:
repeat = False
else:
print("Invalid choice")
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 10
Element inserted at index 1
1. Insert
2. Display
3. Exit
Enter your choice: 2
Values are:
-1
10
-1
-1
-1
-1
-1
-1
-1
-1